Section II Number 1 and 2

Bryan Dion Cellona and Nathalie Faith Ricafort

2026-10-11

II. Analyze the specified Bitcoin and Ethereum return datasets using distribution fitting and normality testing.

1. Find out which probability distribution function best fits Bitcoin's returns for trading data every minute, from January 1, 2012 to September 15, 2026, for Bitcoin quoted in United States dollars or the BTC/USD pair.

Data Preparation

```
btc <- fread("btcusd_1-min_data.csv")

# Convert timestamp
btc$Timestamp <- as.POSIXct(
  as.numeric(btc$Timestamp),
  origin = "1970-01-01",
  tz = "UTC"
)

# Filter dates and clean data
btc <- btc[
  Timestamp >= as.POSIXct("2012-01-01", tz = "UTC") &
  Timestamp < as.POSIXct("2026-09-16", tz = "UTC") &
  is.finite(Close) & Close > 0
]

setorder(btc, Timestamp)

# Calculate one-minute log returns
btc[, log_return := c(NA, diff(log(Close)))]
btc[, gap := c(NA, diff(as.numeric(Timestamp)))]

returns <- btc[
  gap == 60 & is.finite(log_return),
  log_return
]
```

```r
cat("Bitcoin observations:", length(returns), "\n")
```

```
## Bitcoin observations: 7735678
```

```r
cat("First date:", as.character(min(btc$Timestamp)), "\n")
```

```
## First date: 2012-01-01 00:01:00
```

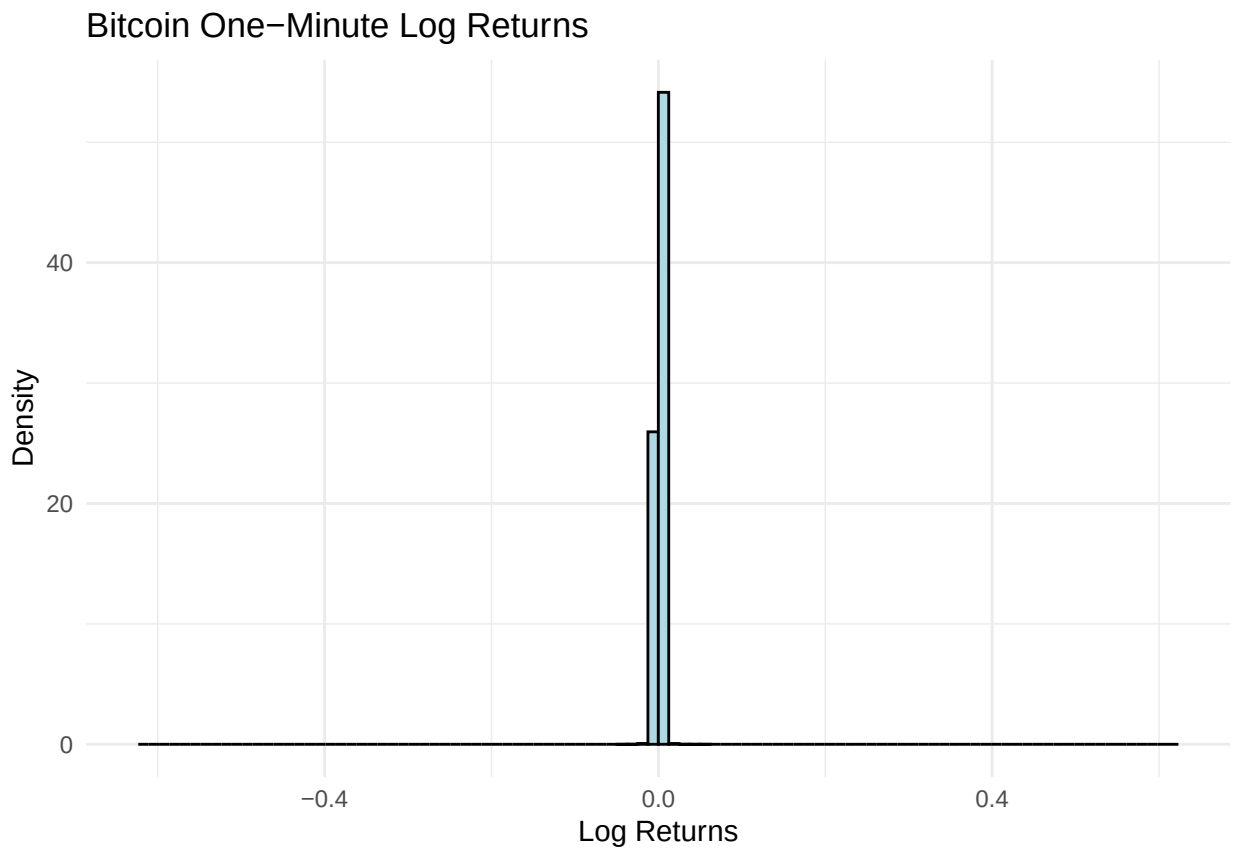
```r
cat("Last date:", as.character(max(btc$Timestamp)), "\n")
```

```
## Last date: 2026-09-15 23:59:00
```

```r
if (length(returns) < 100) {
  stop("Insufficient Bitcoin returns.")
}
```

Histogram of Bitcoin Returns

```r
ggplot(data.frame(returns), aes(x = returns)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 100,
    fill = "lightblue",
    color = "black"
  ) +
  labs(
    title = "Bitcoin One-Minute Log Returns",
    x = "Log Returns",
    y = "Density"
  ) +
  theme_minimal()
```



Distribution Fitting

Four distributions are compared:

1. Normal
2. Student's t
3. Logistic
4. Laplace

A reproducible sample is used to make the fitting process faster.

```
set.seed(123)

x <- sample(
  returns,
  min(30000, length(returns))
)

# Scale returns for numerical stability
x <- x * 10000
n <- length(x)

# Normal distribution
mu_n <- mean(x)
```

```r
sd_n <- sqrt(mean((x - mu_n)^2))

ll_n <- sum(dnorm(x, mu_n, sd_n, log = TRUE))

# Student's t distribution
t_nll <- function(p) {
  mu <- p[1]
  s <- exp(p[2])
  df <- 2 + exp(p[3])

  -sum(dt((x - mu) / s, df, log = TRUE) - log(s))
}

fit_t <- optim(
  c(median(x), log(sd(x)), log(3)),
  t_nll,
  method = "BFGS",
  control = list(maxit = 500)
)

ll_t <- -fit_t$value

# Logistic distribution
logistic_nll <- function(p) {
  mu <- p[1]
  s <- exp(p[2])
  -sum(dlogis(x, mu, s, log = TRUE))
}

fit_l <- optim(
  c(median(x), log(sd(x))),
  logistic_nll,
  method = "BFGS"
)

ll_l <- -fit_l$value

# Laplace distribution
mu_lp <- median(x)
b_lp <- mean(abs(x - mu_lp))

ll_lp <- -n * log(2 * b_lp) -
  sum(abs(x - mu_lp)) / b_lp

# Compare AIC and BIC
results <- data.frame(
  Distribution = c(
    "Normal", "Student's t",
    "Logistic", "Laplace"
  ),
  AIC = c(
    4 - 2 * ll_n,
    6 - 2 * ll_t,
```

```
    4 - 2 * ll_l,
    4 - 2 * ll_lp
  ),
  BIC = c(
    log(n)*2 - 2*ll_n,
    log(n)*3 - 2*ll_t,
    log(n)*2 - 2*ll_l,
    log(n)*2 - 2*ll_lp
  )
)

results <- results[order(results$AIC), ]

print(results, row.names = FALSE)
```

```
##  Distribution      AIC      BIC
##   Student's t 209523.7 209548.6
##       Laplace 215136.5 215153.1
##      Logistic 227351.6 227368.3
##        Normal 248785.9 248802.5
```

Comparison of Fitted Distributions

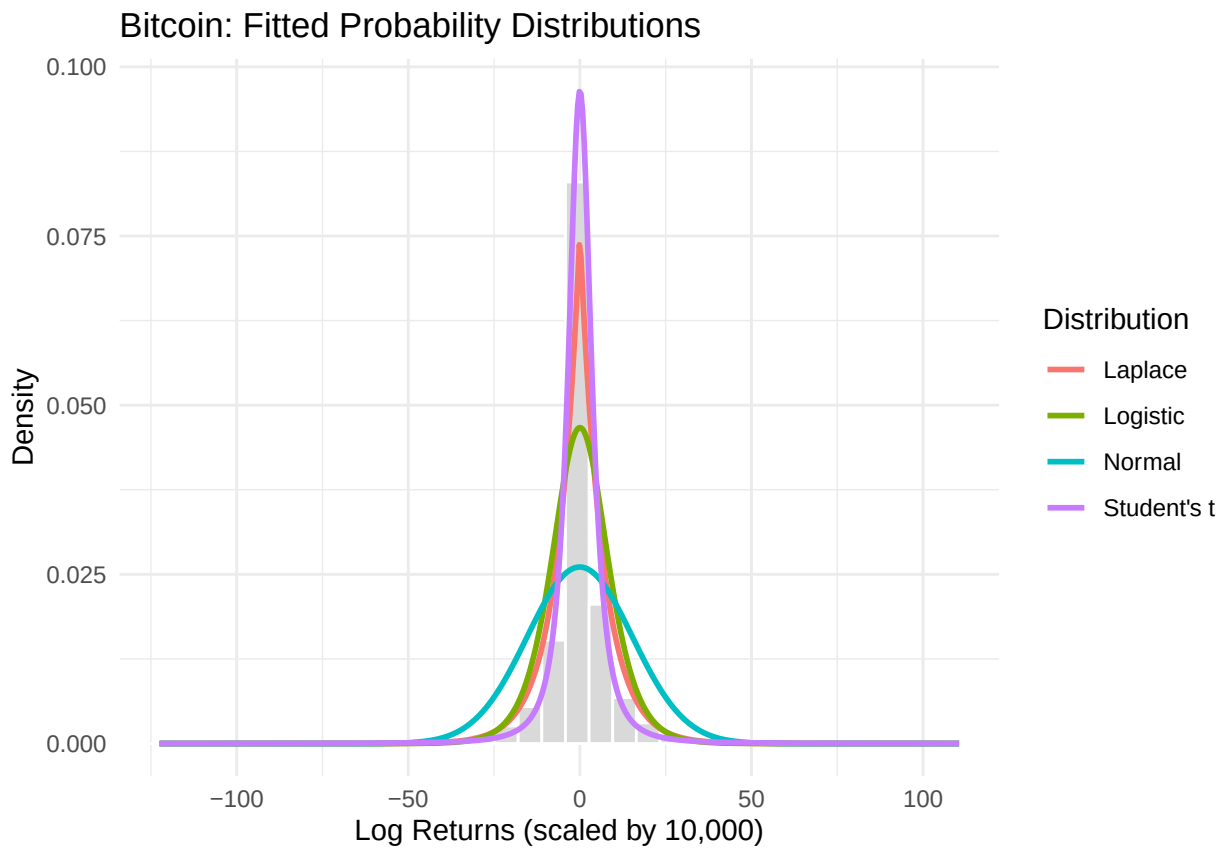
```
# Parameters
mu_t <- fit_t$par[1]
s_t <- exp(fit_t$par[2])
df_t <- 2 + exp(fit_t$par[3])

mu_l <- fit_l$par[1]
s_l <- exp(fit_l$par[2])

# Density curves
xx <- seq(
  quantile(x, 0.001),
  quantile(x, 0.999),
  length.out = 500
)

curves <- data.frame(
  x = rep(xx, 4),
  density = c(
    dnorm(xx, mu_n, sd_n),
    dt((xx - mu_t)/s_t, df_t)/s_t,
    dlogis(xx, mu_l, s_l),
    exp(-abs(xx - mu_lp)/b_lp)/(2*b_lp)
  ),
  Distribution = rep(
    c("Normal", "Student's t", "Logistic", "Laplace"),
    each = length(xx)
  )
)
```

```r
ggplot(data.frame(x), aes(x = x)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 100,
    fill = "gray85",
    color = "white"
  ) +
  geom_line(
    data = curves,
    aes(x = x, y = density, color = Distribution),
    linewidth = 1,
    inherit.aes = FALSE
  ) +
  coord_cartesian(xlim = range(xx)) +
  labs(
    title = "Bitcoin: Fitted Probability Distributions",
    x = "Log Returns (scaled by 10,000)",
    y = "Density"
  ) +
  theme_minimal()
```



Conclusion

```
best <- results$Distribution[1]

cat("Best-fitting distribution:", best, "\n")
```

```
## Best-fitting distribution: Student's t
```

```
cat("Lowest AIC:", results$AIC[1], "\n")
```

```
## Lowest AIC: 209523.7
```

```
cat(
  "Based on the AIC comparison,",
  best,
  "provides the best relative fit among",
  "the four candidate distributions.\n"
)
```

```
## Based on the AIC comparison, Student's t provides the best relative fit among the four candidate dis
```

The distribution with the lowest AIC is selected as the best-fitting model among the four distributions considered. BIC provides an additional comparison.

---

2. Test using Shapiro-Wilk normality test the Ethereum returns for trading data every five minutes, from August 7, 2015 to September 15, 2026.

Data Preparation

```
eth <- fread("ethusd_1min_ohlc.csv")

# Convert timestamp
eth$timestamp <- as.POSIXct(
  as.numeric(eth$timestamp),
  origin = "1970-01-01",
  tz = "UTC"
)

# Filter dates
eth <- eth[
  timestamp >= as.POSIXct("2015-08-07", tz = "UTC") &
  timestamp < as.POSIXct("2026-09-16", tz = "UTC") &
  is.finite(close) & close > 0
```

```
]

setorder(eth, timestamp)

cat("First date:", as.character(min(eth$timestamp)), "\n")
```

```
## First date: 2017-08-16 16:45:00
```

```
cat("Last date:", as.character(max(eth$timestamp)), "\n")
```

```
## Last date: 2026-09-15 23:59:00
```

```
# Convert one-minute data to five-minute data
eth[, interval := floor_date(timestamp, "5 minutes")]

eth5 <- eth[
  , .(price = tail(close, 1)),
  by = interval
]

setorder(eth5, interval)

# Calculate five-minute log returns
eth5[, log_return := c(NA, diff(log(price)))]
eth5[, gap := c(NA, diff(as.numeric(interval)))]

returns5 <- eth5[
  gap == 300 & is.finite(log_return),
  log_return
]

cat("Five-minute returns:", length(returns5), "\n")
```

```
## Five-minute returns: 955382
```

```
if (length(returns5) < 3) {
  stop("Insufficient Ethereum returns.")
}
```

Shapiro-Wilk Normality Test

Hypotheses:

H_0: Ethereum five-minute returns are normally distributed.

H_1: Ethereum five-minute returns are not normally distributed.

Significance level: $\alpha = 0.05$.

```r
set.seed(123)

# Maximum of 5000 observations
eth_sample <- sample(
  returns5,
  min(5000, length(returns5))
)

shapiro_result <- shapiro.test(eth_sample)

print(shapiro_result)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  eth_sample
## W = 0.86462, p-value < 2.2e-16
```

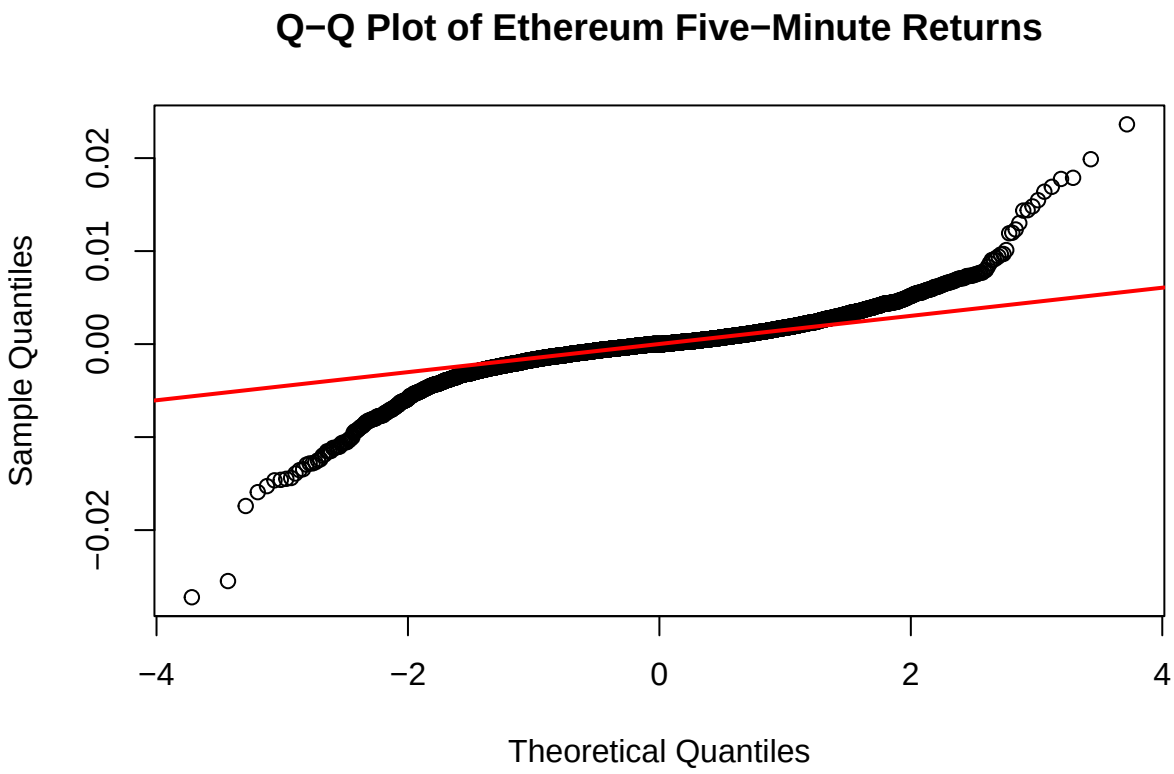
```r
if (shapiro_result$p.value < 0.05) {
  cat("\nDecision: Reject H0.\n")
  cat("The sampled Ethereum returns are not normally distributed.\n")
} else {
  cat("\nDecision: Fail to reject H0.\n")
  cat("There is insufficient evidence against normality.\n")
}
```

```
##
## Decision: Reject H0.
## The sampled Ethereum returns are not normally distributed.
```

Q-Q Plot

```r
qqnorm(
  eth_sample,
  main = "Q-Q Plot of Ethereum Five-Minute Returns"
)

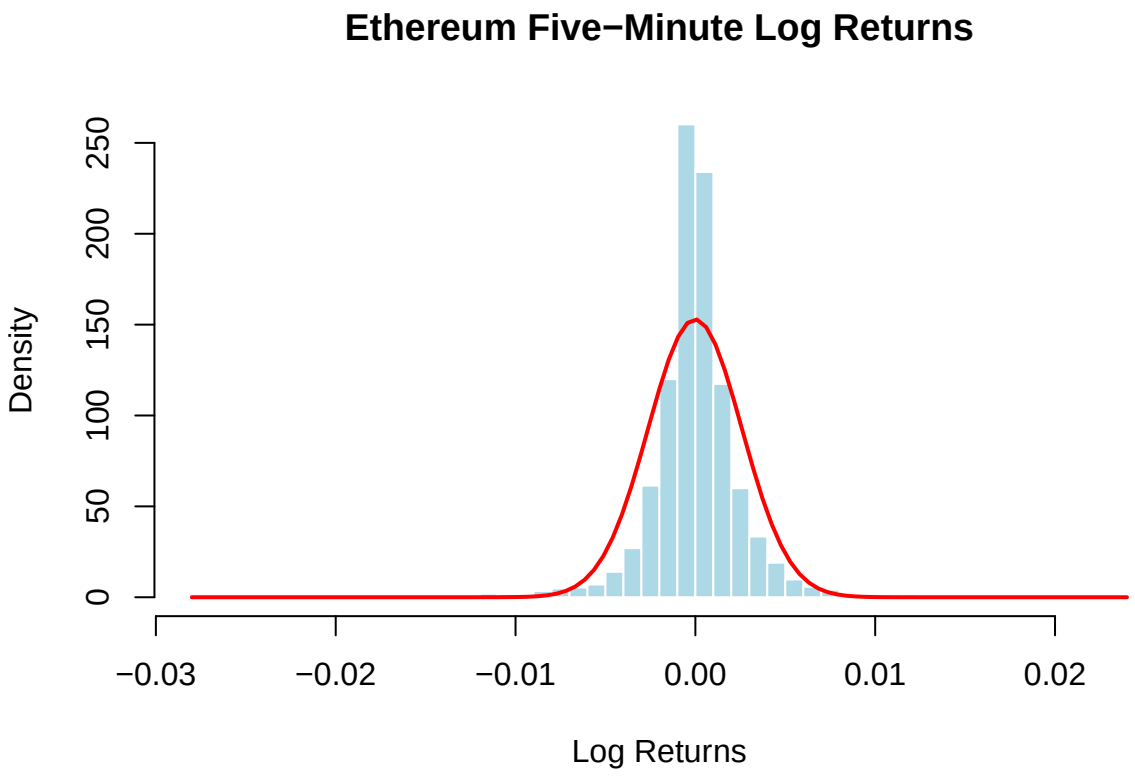
qqline(eth_sample, col = "red", lwd = 2)
```



Histogram with Normal Curve

```
hist(
  eth_sample,
  breaks = 60,
  probability = TRUE,
  col = "lightblue",
  border = "white",
  main = "Ethereum Five-Minute Log Returns",
  xlab = "Log Returns"
)

curve(
  dnorm(
    x,
    mean = mean(eth_sample),
    sd = sd(eth_sample)
  ),
  col = "red",
  lwd = 2,
  add = TRUE
)
```



Conclusion

```
cat("Shapiro-Wilk W statistic:",
    unname(shapiro_result$statistic), "\n")
```

```
## Shapiro-Wilk W statistic: 0.8646156
```

```
cat("P-value:", shapiro_result$p.value, "\n")
```

```
## P-value: 1.914741e-54
```

```
if (shapiro_result$p.value < 0.05) {
  cat(
    "At the 5% significance level,",
    "we reject the null hypothesis.",
    "The sampled Ethereum five-minute",
    "returns are not normally distributed."
  )
} else {
  cat(
    "At the 5% significance level,",
    "we fail to reject the null hypothesis.",
```

```
    "There is insufficient evidence",
    "to reject normality."
  )
}
```

```
## At the 5% significance level, we reject the null hypothesis. The sampled Ethereum five-minute returns
```